

Air University

Department of Computer Science

Introduction to Parallel & Distributed Computing (CS426L)

Lab Report 01: Sequential and Parallel Computation in C#

Student Name	Muhammad Fahad Noor
Registration No.	232448
Section	BSCS-ev-7-C
Instructor	Muhammad Nauman Zafar Hashmi
Date	Sunday, September 13, 2026

Section 1: Environment Setup

The .NET SDK was installed and verified using `dotnet --version`. A test console project (PDC_Lab01) was created and confirmed to print “Welcome to PDC Labs!” successfully.

```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\Lab\Lab1_PDC\CoreInfo>dotnet --version
10.0.401
```

```
Microsoft Windows [Version 10.0.26200.9445]
(c) Microsoft Corporation. All rights reserved.

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\Lab\Lab1_PDC> dotnet run -c Release
Hello, World!
```

Section 2: Task 1 - Sequential Array Summation

Source file: SequentialArraySum.cs

```
using System.Diagnostics;

int size = 10_000_000;
int[] data = new int[size];

// Initialize array before timing
for (int i = 0; i < size; i++)
{
    data[i] = 1;
}

Stopwatch stopwatch = Stopwatch.StartNew();

long sum = 0;

// Sum all elements of the array
for (int i = 0; i < size; i++)
{
    sum += data[i];
}

stopwatch.Stop();

double elapsedMs = stopwatch.Elapsed.TotalMilliseconds;

Console.WriteLine($"Array size: {size}");
Console.WriteLine($"Sum: {sum}");
Console.WriteLine($"Execution time: {elapsedMs} ms");
```

Results

Array Size	Run 1 (ms)	Run 2 (ms)	Run 3 (ms)	Average (ms)
10,000,000	7.2308	5.0826	5.3500	5.8878

```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\Lab\Lab1_PDC>cd C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\Lab\Lab1_PDC\SequentialArraySum

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\Lab\Lab1_PDC\SequentialArraySum>dotnet run -c Release
Array size: 10000000
Sum: 10000000
Execution time: 7.2308 ms

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\Lab\Lab1_PDC\SequentialArraySum>dotnet run -c Release
Array size: 10000000
Sum: 10000000
Execution time: 5.0826 ms

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\Lab\Lab1_PDC\SequentialArraySum>dotnet run -c Release
Array size: 10000000
Sum: 10000000
Execution time: 5.35 ms
```

Section 3: Task 2 - Sequential Matrix Multiplication

Source file: SequentialMatrixMultiplication.cs

```
using System.Diagnostics;

static void InitializeMatrices(double[,] A, double[,] B)
{
    int n = A.GetLength(0);
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            A[i, j] = 1.0;
            B[i, j] = 1.0;
        }
    }
}

static void MultiplySequential(double[,] A, double[,] B, double[,] C)
{
    int n = A.GetLength(0);

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            double sum = 0.0;
            for (int k = 0; k < n; k++)
            {
                sum += A[i, k] * B[k, j];
            }
            C[i, j] = sum;
        }
    }
}

int[] sizes = { 200, 400, 600 };

foreach (int n in sizes)
{
    double[,] A = new double[n, n];
    double[,] B = new double[n, n];
    double[,] C = new double[n, n];

    InitializeMatrices(A, B);

    Stopwatch stopwatch = Stopwatch.StartNew();

    MultiplySequential(A, B, C);

    stopwatch.Stop();

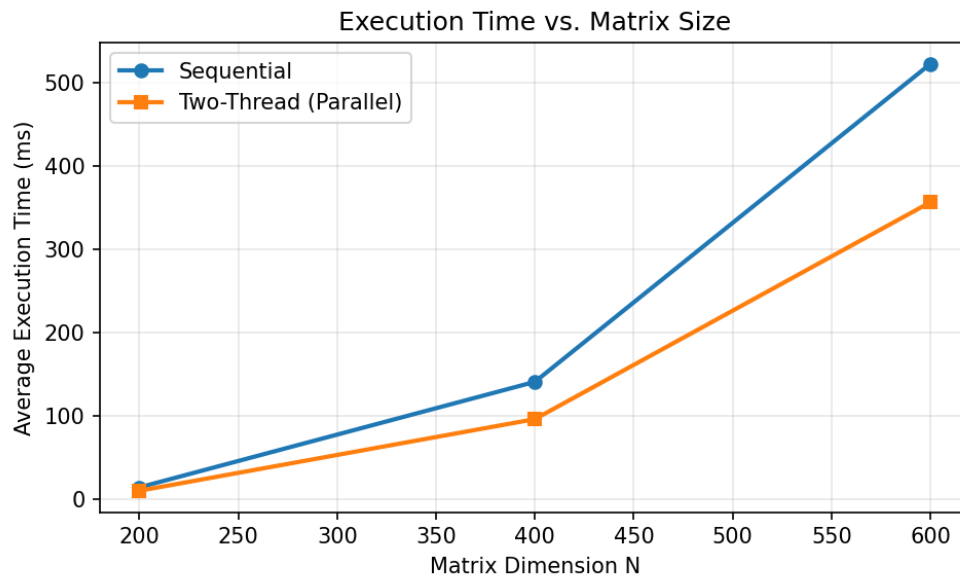
    double elapsedMs = stopwatch.Elapsed.TotalMilliseconds;

    Console.WriteLine($"Matrix size: {n} x {n}");
    Console.WriteLine($"Execution time: {elapsedMs} ms");
}
```

Results

Matrix Size	Run 1 (ms)	Run 2 (ms)	Run 3 (ms)	Average (ms)
200 × 200	14.3305	13.2947	13.2267	13.6173
400 × 400	175.6628	121.0678	124.6919	140.4742
600 × 600	688.2015	435.5080	444.3595	522.6897

Execution-Time Plot



Observation Questions

1. What happens to execution time as the matrix size increases?

Execution time increases sharply and non-linearly as matrix size increases. Going from 200×200 to 600×600 (a 3x increase in dimension) increased the sequential time from about 13.6 ms to about 522.7 ms, nearly a 38x increase, consistent with the $O(N^3)$ cost of dense matrix multiplication.

2. Does doubling the matrix dimension approximately double the execution time? Support your answer using your measured results.

No. Doubling N should increase work by roughly $2^3 = 8x$ for $O(N^3)$ matrix multiplication, not 2x. Going from 200 to 400 (2x the dimension), the sequential time rose from 13.6173 ms to 140.4742 ms, a ~10.3x increase, close to the expected ~8x scaling, and far more than doubling.

3. Why is dense matrix multiplication more computationally demanding than the array summation performed in Task 1?

Array summation performs a single pass over N elements, an $O(N)$ operation. Dense matrix multiplication for an $N \times N$ matrix requires three nested loops, performing $O(N^3)$ multiply-add operations. Even at $N=600$ ($600 \times 600 = 360,000$ elements, far fewer than the 10,000,000-element array in Task 1), matrix multiplication took roughly 89x longer than the array sum, because the total operation count ($N^3 = 216,000,000$) vastly exceeds a simple linear traversal.

```
C:\Users\DELL\Desktop\BSCS-ev-7-C\POC\Lab\Lab1_POC\SequentialArraySum>cd ..\SequentialMatrixMultiplication
C:\Users\DELL\Desktop\BSCS-ev-7-C\POC\Lab\Lab1_POC\SequentialMatrixMultiplication>dotnet run -c Release
Matrix size: 200 x 200
Execution time: 14.3385 ms
Matrix size: 400 x 400
Execution time: 97.6633 ms
Matrix size: 600 x 600
Execution time: 368.2015 ms

C:\Users\DELL\Desktop\BSCS-ev-7-C\POC\Lab\Lab1_POC\SequentialMatrixMultiplication>dotnet run -c Release
Matrix size: 200 x 200
Execution time: 13.2947 ms
Matrix size: 400 x 400
Execution time: 121.0678 ms
Matrix size: 600 x 600
Execution time: 435.588 ms

C:\Users\DELL\Desktop\BSCS-ev-7-C\POC\Lab\Lab1_POC\SequentialMatrixMultiplication>dotnet run -c Release
Matrix size: 200 x 200
Execution time: 13.2267 ms
Matrix size: 400 x 400
Execution time: 124.6919 ms
Matrix size: 600 x 600
Execution time: 444.3595 ms
```

Section 4: Task 3 - Two-Thread Matrix Multiplication

Source file: ParallelMatrixMultiplication.cs

```
using System.Diagnostics;
using System.Threading;

static void InitializeMatrices(double[,] A, double[,] B)
{
    int n = A.GetLength(0);
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            A[i, j] = 1.0;
            B[i, j] = 1.0;
        }
    }
}

class MatrixWorker
{
    private readonly double[,] A;
    private readonly double[,] B;
    private readonly double[,] C;
    private readonly int startRow;
    private readonly int endRow;

    public MatrixWorker(double[,] A, double[,] B, double[,] C, int startRow, int endRow)
    {
        this.A = A;
        this.B = B;
        this.C = C;
        this.startRow = startRow;
        this.endRow = endRow;
    }

    public void Run()
    {
        int n = A.GetLength(0);

        for (int i = startRow; i < endRow; i++)
        {
            for (int j = 0; j < n; j++)
            {
                double sum = 0.0;

                for (int k = 0; k < n; k++)
                {
                    sum += A[i, k] * B[k, j];
                }

                C[i, j] = sum;
            }
        }
    }
}

int[] sizes = { 200, 400, 600 };

foreach (int n in sizes)
{
    double[,] A = new double[n, n];
    double[,] B = new double[n, n];
    double[,] C = new double[n, n];
}
```

```

InitializeMatrices(A, B);

int midpoint = n / 2;

MatrixWorker worker1 = new MatrixWorker(A, B, C, 0, midpoint);
MatrixWorker worker2 = new MatrixWorker(A, B, C, midpoint, n);

Thread t1 = new Thread(new ThreadStart(worker1.Run));
Thread t2 = new Thread(new ThreadStart(worker2.Run));

Stopwatch stopwatch = Stopwatch.StartNew();

t1.Start();
t2.Start();

t1.Join();
t2.Join();

stopwatch.Stop();

double elapsedMs = stopwatch.Elapsed.TotalMilliseconds;

Console.WriteLine($"Matrix size: {n} x {n}");
Console.WriteLine($"Execution time: {elapsedMs} ms");
}

```

Results

Matrix Size	Run 1 (ms)	Run 2 (ms)	Run 3 (ms)	Average (ms)
200 × 200	10.3503	9.0847	9.0979	9.5110
400 × 400	106.4708	79.1115	101.2987	95.6270
600 × 600	383.4948	264.3178	421.3452	356.3859

Sequential vs. Two-Thread Comparison

Speedup = Sequential Execution Time ÷ Parallel Execution Time

Matrix Size	Sequential Time (ms)	Two-Thread Time (ms)	Speedup
200 × 200	13.6173	9.5110	1.4317
400 × 400	140.4742	95.6270	1.4690
600 × 600	522.6897	356.3859	1.4666

Observation Questions

1. Did the two-thread implementation execute faster than the sequential implementation for every problem size?

Yes. The two-thread version was faster at all three sizes: 9.5110 ms vs 13.6173 ms at 200×200, 95.6270 ms vs 140.4742 ms at 400×400, and 356.3859 ms vs 522.6897 ms at 600×600.

2. For which problem size was the benefit of using two threads most visible?

The speedup was fairly consistent across sizes (approximately 1.43–1.47x), with the 400×400 case showing the highest measured speedup (1.4690x). The absolute time saved, however, was largest at 600×600 (about 166 ms saved) simply because the total workload was larger.

3. Why might the measured speedup be less than 2 even though two threads are being used?

Speedup of exactly 2x assumes perfectly parallel work with zero overhead. In practice, thread creation, OS scheduling, context switching, and shared memory/cache contention between the two threads (both reading the same A and B matrices) all consume time that does not contribute to useful computation, reducing the achievable speedup below the theoretical maximum.

4. What overhead is introduced when threads are created, started, scheduled, and joined?

Creating a Thread object allocates OS-level resources (a stack, thread control block). Start() requires the OS scheduler to allocate the new thread a core/time-slice, which involves a context switch. While running, the two threads may contend for shared CPU cache lines and memory bandwidth. Join() forces the main thread to block and wait, and the OS must again context-switch back once the worker's finish. All of this is pure overhead not present in the single-threaded sequential version.

5. Why is dividing the result matrix by rows a convenient method of splitting this problem?

Each row of the result matrix C is computed entirely independently of every other row computing $C[i,*]$ only requires reading row i of A and all of B, and writes only to row i of C. Splitting by rows therefore requires no synchronization between threads: each thread reads shared, unmodified input matrices and writes to a disjoint set of output rows, eliminating any risk of a race condition without needing locks.

```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\Lab\Lab1_PDC\ParallelMatrixMultiplication>dotnet run -c Release
Matrix size: 200 x 200
Execution time: 10.3503 ms
Matrix size: 400 x 400
Execution time: 106.4708 ms
Matrix size: 600 x 600
Execution time: 383.4948 ms

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\Lab\Lab1_PDC\ParallelMatrixMultiplication>dotnet run -c Release
Matrix size: 200 x 200
Execution time: 9.0847 ms
Matrix size: 400 x 400
Execution time: 79.1115 ms
Matrix size: 600 x 600
Execution time: 264.3178 ms

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\Lab\Lab1_PDC\ParallelMatrixMultiplication>dotnet run -c Release
Matrix size: 200 x 200
Execution time: 9.0979 ms
Matrix size: 400 x 400
Execution time: 101.2987 ms
Matrix size: 600 x 600
Execution time: 421.3452 ms
```

Section 5: Task 4 - Core Count, Clock Speed, and Why Parallelism Matters

Source file: CoreInfo.cs

```
using System.Diagnostics;

Console.WriteLine($"Logical processors available: {Environment.ProcessorCount}");

// Windows: query physical cores, logical processors, and clock speed via PowerShell
var psi = new ProcessStartInfo("powershell",
    "-Command \"Get-CimInstance Win32_Processor | Select-Object Name, NumberOfCores, NumberOfLogicalProcessors, MaxClockSpeed\"");
{
    RedirectStandardOutput = true,
    UseShellExecute = false
};

using var process = Process.Start(psi);
string output = process!.StandardOutput.ReadToEnd();
Console.WriteLine(output);
```

Hardware Data

Property	Value
Environment.ProcessorCount (C#)	8
Physical cores	4 (Intel Core i5-10400H)
Logical processors	8
Base clock speed	2.60 GHz
Max / turbo clock speed	2592 MHz reported by Win32_Processor (~2.6 GHz; actual turbo boost for this CPU model is higher and not exposed by this API)

Discussion Questions

1. Using your recorded core count and the speedup you measured in Task 3, explain in your own words why adding cores rather than raising clock frequency became the practical way to increase computing power (consider power consumption and heat dissipation).

My machine has 4 physical cores (8 logical, via hyperthreading) running at roughly 2.6 GHz. Using just 2 of those cores gave a consistent ~1.45x speedup on matrix multiplication. Power consumed by a CPU core scales roughly with the cube of its clock frequency, while performance scales only linearly with frequency so doubling clock speed would require roughly 8x the power and generate far more heat than a chip can practically dissipate. Adding cores instead lets performance scale by running more work simultaneously without paying that cubic power penalty, which is why manufacturers shifted from chasing higher GHz to packing in more cores once clock speeds plateaued in the mid-2000s.

2. What would you expect to happen to execution time if the matrix multiplication were divided among a number of threads equal to Environment.ProcessorCount instead of just two? What factors could prevent the speedup from scaling linearly with core count?

Using 8 threads instead of 2 would likely reduce execution time further, though not by a full additional 4x on top of the 2-thread result. Diminishing returns would appear because of: increased overhead from creating/scheduling more threads; more contention for shared memory bandwidth and CPU cache as more cores simultaneously access the A and B matrices; the fact that 4 of the 8 logical processors are hyperthreaded siblings sharing physical core resources rather than being fully independent cores; and any serial portions of the program (initialization, timer setup) that cannot be parallelized at all, per Amdahl's Law.

3. Relate your Task 1 (array summation) timing to your Task 2 (matrix multiplication) timing and the core count of your machine. Which of the two problems would benefit more from being split across many cores, and why?

Task 1 (summing 10,000,000 elements) completed in under 6 ms an $O(N)$ operation that is already memory-bandwidth-bound rather than compute-bound, so splitting it across many cores would mostly just add thread overhead with little computational work to actually parallelize per thread. Task 2's matrix multiplication ($O(N^3)$) is far more compute-intensive 522.7 ms at just 600×600 and each output element's computation is independent, giving it much more genuine parallel work to distribute. Matrix multiplication would benefit far more from many cores, because there is substantially more CPU-bound work to divide relative to the fixed overhead of managing threads.

```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\Lab\Lab1_PDC\ParallelMatrixMultiplication>cd ..\CoreInfo
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\Lab\Lab1_PDC\CoreInfo>dotnet run -c Release
Logical processors available: 8

Name                                     NumberOfCores NumberOfLogicalProcessors MaxClockSpeed
----                                     -
Intel(R) Core(TM) i5-10400H CPU @ 2.60GHz 4                      8                      2592
```